Programming Assignment 1.5 - Faster Matrix Multiplication

1. Introduction

In Programming Assignment 1(PA1), you implemented a simple GEMM kernel in CUDA. This assignment is a direct continuation of PA1, hence "1.5". This time you will optimize your GEMM kernel by using Tensor cores of the GPU.

Tensor cores are simply matrix multipliers. They take a matrix A of size **mxk**, and a matrix B of size **kxn**, and calculate the output matrix C of size **mxn**. They are fast because the entire matrix multiplication takes one instruction in hardware. This instruction is commonly referred to as the Matrix-multiply-accumulate(MMA) instruction.

There are multiple ways to program tensor cores in CUDA kernels. In this assignment, you will use the C++ MMA wrappers that we provide. Simply put, these wrappers are CUDA C++ device functions that you can call from your kernel[1]. You will write GEMM kernels that use these wrappers, time them and compare them to your kernels from PA1.

2. Using Tensor Cores

You only need to concern yourself with the following function defined in the `mma_intrinsics.cuh` file,

```
__device__ void mma_m16n8k16_f16_f16_smem_row_col(__half *A, __half
*B, float *C, int ldmA=16, int ldmB=16, int ldmC=8)
```

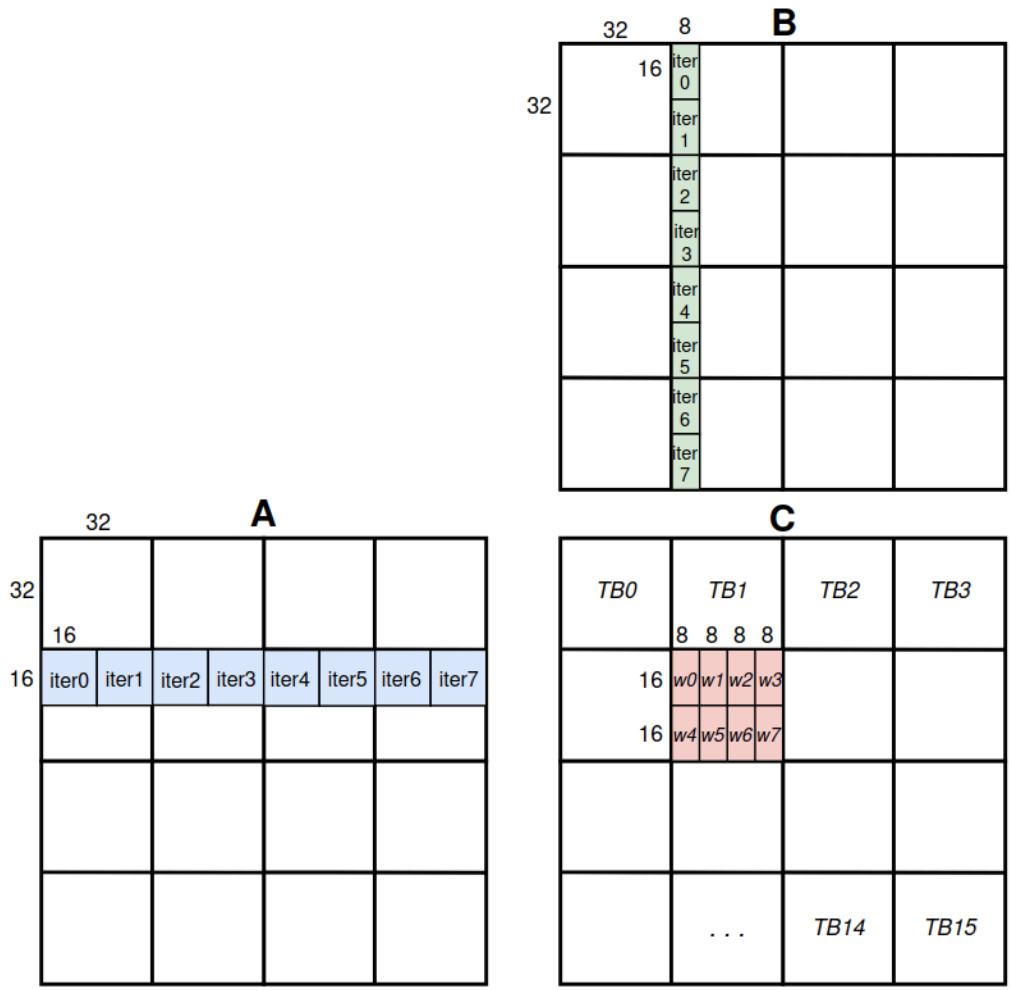
It may look verbose, but all it does is take two fp16 pointers(__half), which represent the A and B matrices, and use tensor cores to perform matrix multiplication and produce the fp32 output matrix C. Tensor cores do not support fp32 matrix multiplication, so we will use fp16 input matrices, but the output will remain fp32. The last three parameters are leading dimensions for the **matrices in shared memory**. Following conditions must be met for proper operation of this function,

1. The matrix A is in **shared memory** in row-major format. Dimensions of input A should be 16x16.
2. The matrix B is in **shared memory** in column-major format. Dimensions of input B should be 16x8
3. The output matrix will be written to C, which should point to **shared memory**. Dimensions of output C are 16x8.

[1]A "device function" is a GPU kernel that can only be called from another GPU kernel, hence the name "device". A "global function" is also a GPU kernel, but it can be launched from the CPU. Device functions are declared using the "__device__" keyword, globals are declared using the "__global__" keyword.

As you can see, you first need to move the input matrices from global memory to shared memory before you can call this function. Once you move the data to shared memory, you can call this function to multiply a 16x16 matrix (A) with a 16x8 matrix (B).

MMA instructions are warp-wide instructions, meaning that all 32 threads in the warp work cooperatively to execute the instruction. To properly utilize tensor cores, it is imperative to think in terms of warps and thread blocks(TB). Each warp should calculate a 16x8 subtile of the output matrix. Suppose that each TB has 8 warps in it, therefore each TB can calculate a 32x32 tile of the output matrix. See the figure below for a visualization,



For simplicity, in the figure, all three matrices are 128x128. Since each TB calculates a 32x32 tile, we need 16 TBs to calculate the entire C matrix. Then, within each TB, each warp calculates a 16x8 subtile. To calculate a 16x8 subtile, each warp will have to perform 8 MMA operations, meaning 8 iterations. In each iteration, the warp will copy input subtiles to shared memory, perform an MMA, and accumulate the results in shared memory. In the figure, the subtiles that warp 0 will load in each iteration are highlighted. The number of iterations depends on the input matrix size, specifically the **K** dimension, which is the number of columns of A and rows of B. The wider the A matrix(and therefore the taller the B matrix), the more iterations need to be done. Specifically, the number of iterations is equal to **K/16**.

You need to allocate enough shared memory so that all the warps in the TB can copy the input subtiles that they need, and accumulate the output subtile C. For the example shown above, each warp needs a 16x16 subtile for A, a 16x8 subtile for B, and a 16x8 subtile for C to accumulate the results. Since each TB has 8 warps, you'd need to allocate 8 times the shared memory each warp needs.

In short, your new GEMM kernel should do the following,

1. For each iteration,
    a. Load input subtiles from global memory to shared memory
        i. Let each thread in a warp copy a row from A (i.e. 16 elements) and a column from B (i.e. 16 elements) to shared memory.
    b. Perform MMA using the `mma_m16n8k16_f16_f16_smem_row_col` device function
2. Store the results back to global memory before exiting.

Here are some tips that might help you,

- The entire tensor core kernel can be written in ~60 lines of code. Keep it simple.
- Do not forget to initialize the shared memory for C to zeroes.
- You can use `__syncwarp()` and `__synchthreads()` CUDA functions to synchronize threads in a warp or in a block, respectively.
- CUDA supports `printf()`. You can use it to debug your code, although it is not good practice. Ideally you should use `cuda-gdb`.
- Thread block and tile dimensions we used for the example above are arbitrary. You do not have to use the exact same dimensions. Your thread blocks can have 4, 8, 12 etc. warps. This is up to you.

3. Task Description

Your task is to use the MMA wrappers to perform GEMM. Use the wrappers to write CUDA kernels for GEMM. Similar to PA1, you also need to write the host code to read/initialize the inputs, allocate device memory and launch the kernel. Prepare a 1-page report briefly describing how your host code and the kernel work. Measure the kernel execution time for your new tensor core kernel and compare it to your GEMM kernel from PA1. How much faster is your tensor core kernel? You are **not** allowed to use any CUDA libraries(CUTLASS, CuDNN etc.).

4. Constraints

Problem size: **64 <= M,N,K <= 4096**
M, N, K are guaranteed to be integer powers of 2.
Input matrices will be in **fp16** format, i.e. `__half`.
Output matrix will be in **fp32** format, i.e. `float`.
To simplify things, you only need to support **row-major** format for A, and **column-major** format for B. The output matrix **C** will be stored **row-major** in memory. More formally,

{T} x {N} = T

where, **T** means row-major, and **N** means column-major.

5. Materials, Testing and Grading

Similar to PA1, template CUDA(.cu) files and a Makefile are provided in a zip file in Moodle. In the zip file, you will find two CUDA files and a Makefile to build them. The `mma_instrinsics.cuh` file is also included in the zip file. You simply need to fill out the CUDA files, similar to PA1.

- `main.cu` is where you read/initialize inputs, allocate device memory, launch kernels and time them. You can compile this file by running "`make main`".
- `launchStudentKernel.cu` has a "`launchStudentKernel()`" function that you will implement, similar to PA1. You can simply copy your kernel code from `main.cu`, paste it in the file, and launch it in the `launchStudentKernel()` function. You can compile the code by running "`make pa1p5_testbench`". This will compile the code, and link it with the testbench binary in the shared directory. It will produce the "pa1p5_testbench" binary in your directory, which you can run to test your kernels. Example: "`./pa1p5_testbench 256 256 128`". This will launch the testbench with m256n256k128 problem size where A is row-major and B is column-major. It will invoke your kernel, and a CUTLASS kernel, and compare their outputs. It will print "`[SUCCESS]`" if the outputs match, "`[FAIL]`" otherwise.

We will provide you with multiple sets of small input-output pairs in .txt format to ease testing. Your program can read the inputs, calculate the outputs and compare them with the expected output. You **do not have** to use these input-output references, they are simply provided for convenience. Feel free to generate your own tests and use them.
We will test your program on Hydra servers, so make sure you get the expected outputs there. You will lose points if your code works on your personal computer, but not on Hydra. See here if you are not familiar with ECE Hydra servers.

You can run "`make grade`" to run the grading script which uses the testbench to run several matrix multiplications and checks your results. We will use this exact same script to determine your grades, so you can basically self-grade before you submit. You get 80 points if you pass all the tests, and 20 points for your report, totaling 100.

6. Cheating and LLMs

- You are supposed to do this assignment on your own. You can have discussions with your classmates, but we expect you to write your own code at the end of the day.

- Do not submit AI generated code.

Violation of these rules may result in zero marks and disciplinary action.